MetroMitra – KOLKATA METRO NAVIGATION SYSTEM

A PROJECT REPORT

Submitted by

ANUSKA DATTA (25BCS10364)
BHAVYA SHARMA (25BCS12027)
KANIKA SAINI (25BCS12309)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

NOVEMBER 2026

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

BONAFIDE CERTIFICATE

Certified that this project report entitled “MetroMitra – Kolkata Metro Navigation System” is the bonafide work of Anuska Datta (25BCS10364), Bhavya Sharma (25BCS12027) and Kanika Saini (25BCS12309), who carried out the project work as part of the Advanced Data Structures and Algorithms (ADSA) course.

Supervisor: _____

Academic Designation: _____

Department: _____

Signature of the Supervisor: _____

Name and Signature of the Head of Department: _____

Submitted for the project viva-voce examination held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

Section	Page
List of Figures	4
List of Tables	4
List of Standards	5
Abstract	6
Graphical Abstract	7
Abbreviations	8
Symbols	8
Chapter 1. Introduction	9
1.1 Need / contemporary issue	9
1.2 Identification of problem	9
1.3 Identification of tasks	10
1.4 Timeline	10
1.5 Organization of report	10
Chapter 2. Literature Review / Background Study	11
2.1 Timeline of reported problem	11
2.2 Existing solutions	11
2.3 Bibliometric / comparative analysis	12
2.4 Review summary	12
2.5 Problem definition	12
2.6 Goals / objectives	12
Chapter 3. Design Flow / Process	13
3.1 Feature evaluation	13
3.2 Design constraints	13
3.3 Feature finalization	14
3.4 Alternative design flows	14
3.5 Design selection	14
3.6 Implementation methodology	15
Chapter 4. Results Analysis and Validation	16
4.1 Implementation of solution	16
Chapter 5. Conclusion and Future Work	18
References	19
Appendix	20
User Manual	21

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

LIST OF FIGURES

Figure	Caption
Figure 3.1	Kolkata Metro network map used as a route-network reference
Figure 3.2	MetroMitra routing and data-processing workflow
Figure 4.1	Dataset: station-to-station connections
Figure 4.2	Dataset: interchange records
Figure 4.3	Dataset: metro lines
Figure 4.4	Dataset: station master records
Figure 4.5	Terminal output – Dakshineswar to Howrah Maidan
Figure 4.6	Terminal output – Mahatma Gandhi Road to Bengal Chemical
Figure 4.7	MetroMitra website network summary
Figure 4.8	MetroMitra website journey planner and route result
Figure 4.9	Verified output of the fewest-interchange extension
Figure A.1	Project source and data folder structure

LIST OF TABLES

Table	Caption
Table 1.1	Project task schedule
Table 2.1	Comparison of route-planning approaches
Table 3.1	Functional feature evaluation
Table 3.2	Design constraints
Table 4.1	Dataset summary
Table 4.2	Test cases and observed results
Table 4.3	Complexity summary

LIST OF STANDARDS

The project is an academic software prototype. The following language and data-format specifications are relevant to its implementation. No claim of formal certification or compliance testing is made.

Standard / specification	Publishing agency	Relevance to project	Where used
ISO/IEC 14882:2017 (C++17)	ISO / IEC	Defines the C++ language features used by the console implementation, including standard containers, tuples and structured bindings.	C++ implementation and compilation
RFC 4180 – Common Format and MIME Type for CSV Files	IETF	Provides a commonly referenced format description for comma-separated tabular data. The project uses simple CSV records with headers.	Dataset files in Chapter 4
HTML Living Standard	WHATWG	Defines the markup platform used to structure the browser interface.	web/index.html
CSS specifications	W3C	Define styling and layout rules used by the website interface.	web/styles.css
ECMAScript specification	ECMA International	Defines JavaScript language behaviour used for data loading, route rendering and form interactions.	web/app.js

ABSTRACT

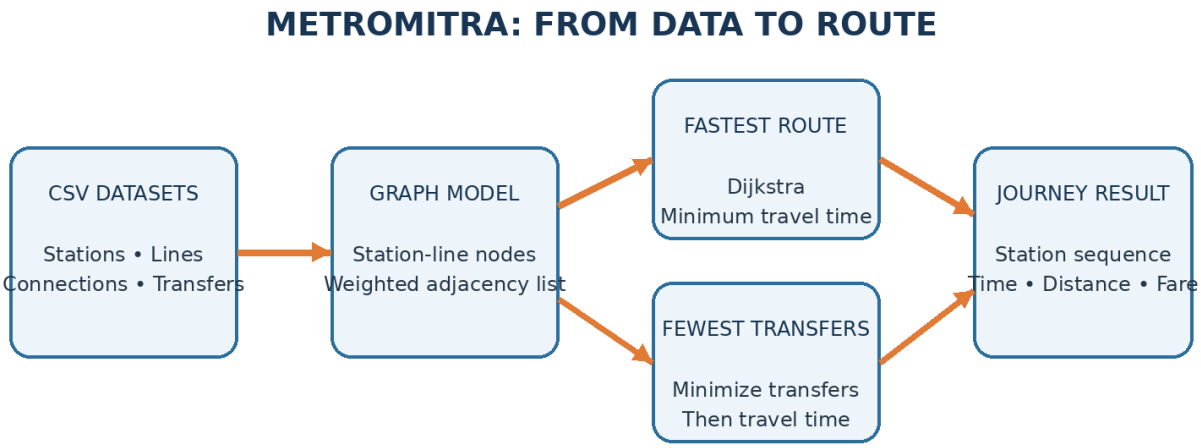
MetroMitra is an academic Kolkata Metro navigation prototype developed to demonstrate the practical use of graph data structures and shortest-path algorithms. The system represents the metro network as a weighted graph. Each graph node represents a station-line combination, while normal station connections and inter-line transfers are represented by weighted edges. The project stores station, line, connection and interchange information in separate CSV files.

The existing route-planning functionality uses Dijkstra's shortest-path algorithm with travel time as the edge weight. It produces a station sequence and calculates estimated travel time, distance and fare. A browser-based interface built with HTML, CSS and JavaScript provides station suggestions, route planning, route display and network statistics.

An additional algorithmic extension, implemented in the supplied contribution version of the C++ source, introduces lexicographic route selection: it first minimizes the number of interchanges and then minimizes travel time among routes with the same interchange count. The updated source was compiled successfully with g++ using the C++17 standard and executed against the supplied dataset. The report distinguishes the prototype's simulated fare and dataset estimates from official live metro information.

The supplied dataset contains 55 station records, five line records, 52 normal connection records and four directional interchange records. The work demonstrates graph modelling, adjacency lists, priority queues, file handling, algorithm design, testing and a simple web interface.

GRAPHICAL ABSTRACT



Graphical Abstract: data ingestion, graph construction, route algorithms and journey output.

The diagram summarizes the processing pipeline. Both route algorithms operate on the same graph representation but optimize different priorities.

ABBREVIATIONS

Abbreviation	Meaning
ADSA	Advanced Data Structures and Algorithms
CSV	Comma-Separated Values
CSE	Computer Science and Engineering
GUI	Graphical User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
UID	University Identification Number
INF	A large sentinel value representing infinity in shortest-path initialization
ID	Identifier

SYMBOLS

Symbol / term	Meaning
V	Set of graph vertices / station-line nodes
E	Set of graph edges
d[v]	Best known route cost to vertex v
w(u,v)	Weight of the edge from u to v
T	Total estimated travel time
D	Total estimated travel distance
k	Number of interchange transitions
O(V + E)	Linear graph traversal bound
O((V + E) log V)	Typical binary-heap Dijkstra time bound

CHAPTER 1. INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

Urban rail systems serve large numbers of commuters and visitors. A passenger planning a journey may need to identify the correct origin and destination, understand which stations connect, determine whether a line change is required and estimate the time and cost of travel. A route description that only lists station names may be insufficient when multiple lines meet at an interchange.

MetroMitra addresses this information need through a small, dataset-driven navigation prototype for the Kolkata Metro network. The project is developed as an academic solution rather than a commissioned system for a named client. Its intended users are students or passengers exploring routes using the supplied network dataset. No user survey or measured passenger study was supplied; therefore, the need is framed as a practical navigation use case rather than a statistically validated market claim.

1.2 Identification of Problem

The broad problem is that metro network information is distributed across station, line, connection and interchange records, making it difficult to manually determine a valid journey and compare route choices. A user needs a consistent way to identify a connected station sequence and understand the travel implications of the selected path. The problem definition does not assume a particular algorithm or interface as its solution.

1.3 Identification of Tasks

- Study the metro network data and identify entities and relationships.
- Organize station, line, connection and interchange records in CSV files.
- Represent station-line combinations as graph vertices and travel links as weighted edges.
- Implement and validate a fastest-route algorithm using Dijkstra's method.
- Implement a second route objective that prioritizes fewer interchanges and then travel time.
- Calculate journey distance, estimated time and simulated fare.
- Develop and test a browser interface for selecting stations and viewing route results.
- Document test cases, limitations, operation steps and future improvements.

1.4 Timeline

Phase	Activities	Indicative sequence
1	Problem understanding and dataset organization	Project initiation
2	Graph representation and C++ implementation	Core development
3	Website structure, styling and JavaScript integration	Interface development
4	Route testing and output verification	Validation
5	Fewest-interchange extension and report preparation	Enhancement and documentation

The sequence above is a phase-based project timeline. Exact calendar start and completion dates were not provided.

1.5 Organization of the Report

Chapter 1 introduces the navigation problem, tasks and project plan. Chapter 2 reviews graph-based route planning and compares relevant approaches. Chapter 3 explains the selected architecture, constraints, alternatives and implementation methodology. Chapter 4 documents the implementation, dataset, screenshots and validation results. Chapter 5 concludes the

work and identifies future improvements. References list the technical sources and specifications consulted. The appendix and user manual provide supporting project information and execution steps.

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Shortest-path computation is a foundational problem in graph theory and computer science. Dijkstra published an influential shortest-path method in 1959. The method is widely taught and applied to weighted networks where edge costs are non-negative. Modern journey planners extend this general graph-search idea with transport-specific constraints such as transfer penalties, service schedules, accessibility and real-time conditions.

For this academic prototype, the network is static: travel time, distance and interchange duration are read from CSV records. It does not model live train positions, waiting-time variation, service disruptions or timetable departures.

2.2 Existing Solutions

Common route-planning approaches include manual route maps, fixed route tables, breadth-first search for unweighted networks, and weighted shortest-path algorithms such as Dijkstra. Commercial journey planners may additionally use live service feeds, timetable data, accessibility information and operational alerts. MetroMitra focuses on the core graph problem and a transparent local dataset.

2.3 Bibliometric / Comparative Analysis

A quantitative bibliometric study requires a defined literature database, search strategy and publication corpus. No such corpus was supplied for this project. In accordance with the report format, this section therefore provides a comparative feature analysis of relevant algorithmic approaches rather than inventing publication counts.

Approach	Key feature	Effectiveness	Limitation
Manual route map	Human interpretation of lines and interchanges	Useful for visual overview	Slow for comparing many alternatives; no automatic time/fare calculation
Breadth-First Search	Minimizes number of edges in an unweighted graph	Simple when all edges have equal cost	Does not optimize weighted travel time
Dijkstra's algorithm	Minimizes sum of non-negative edge weights	Suitable for fastest estimated route in this weighted network	Optimizes one scalar cost; does not inherently prioritize transfer count
Lexicographic Dijkstra extension	Minimizes interchange count first, then time	Supports a passenger preference for fewer line changes	May choose a longer journey time if it avoids a transfer; relies on dataset accuracy

2.4 Review Summary

The review supports modelling the metro system as a weighted graph. Dijkstra's algorithm is appropriate for the fastest-route objective when all travel and transfer weights are non-negative. A second lexicographic cost objective can represent the preference for fewer transfers, using a pair (interchange count, travel time). The two objectives provide complementary route choices rather than treating one route as best for every passenger.

2.5 Problem Definition

The project must accept an origin and destination station, determine whether a route exists in the supplied network, and return a station-line path. It must calculate total travel time and distance and estimate a fare using the academic rule defined in the dataset documentation. The enhanced version must also return a path that minimizes interchange count, using travel time as the secondary criterion. The project does not claim to provide official fares, real-time train information or complete coverage of every operational Kolkata Metro station.

2.6 Goals / Objectives

- Load and organize the supplied 55 station records, five line records, 52 connection records and four directional interchange records.
- Construct a graph whose vertices distinguish station-line combinations.
- Find a minimum-travel-time route using Dijkstra's algorithm.
- Find a minimum-interchange route and use travel time to break ties.
- Display route stations and interchange information in the console and web interface where supported.
- Calculate distance, estimated time and simulated fare.
- Validate the implementation with representative source-destination pairs and invalid-input cases.

CHAPTER 3. DESIGN FLOW / PROCESS

3.1 Evaluation & Selection of Specifications / Features

Feature	Purpose	Decision
Station lookup	Resolve entered station names to station IDs	Selected
Fastest route	Minimize total estimated travel time	Selected
Fewest interchanges	Minimize line changes; time is tie-breaker	Selected as enhancement
Distance and time	Summarize journey cost	Selected
Fare estimate	Provide an academic fare approximation	Selected with clear disclaimer
Live train tracking	Use current train location and service data	Not selected: no live feed supplied
Interactive geographic map	Show routes on a live map	Future scope; static network map used as reference

3.2 Design Constraints

Constraint area	Consideration
Data	Route results depend on the completeness and correctness of the four CSV files.
Algorithm	Dijkstra's algorithm assumes non-negative edge weights; supplied travel and transfer times are non-negative.
Fare	Fare is simulated as ceil(distance / 2) × ₹5 and is not an official Kolkata Metro fare.
Platform	C++ console build uses a C++17-capable compiler; website uses a modern browser.
Network coverage	The dataset contains only 55 station records, five line records and 52 connection records.
Safety / ethics	The website should not be presented as an official or real-time transit authority service.
Maintainability	Keeping data in CSV files separates network records from program logic.

No formal safety certification, regulatory approval or accessibility audit was supplied. These remain considerations for a production-grade public service.

3.3 Analysis of Features and Finalization Subject to Constraints

The final scope prioritizes features that can be implemented and validated using the available data. Fastest routing, fewest-interchange routing, station selection and journey summaries are retained. Real-time tracking and live fare integration are excluded because the project has no live data source or official fare feed. The website provides a route-planning interface, but the supplied JavaScript currently implements the fastest route only; the new fewest-interchange algorithm is verified in the C++ contribution source and should be integrated into the web interface as future work.

3.4 Design Flow

Alternative Design A – Console-first graph application: read CSV files, build the graph, accept station names, run Dijkstra and print the result. This approach is simple to test and demonstrates ADSA concepts, but offers limited usability.

Alternative Design B – Data-driven web interface supported by graph logic: load CSV datasets in JavaScript, construct a station-line graph, plan a route and render journey details in a browser. The C++ implementation remains the academic algorithm demonstration and is extended with a second route objective. This approach improves accessibility and presentation while retaining a clear algorithmic core.

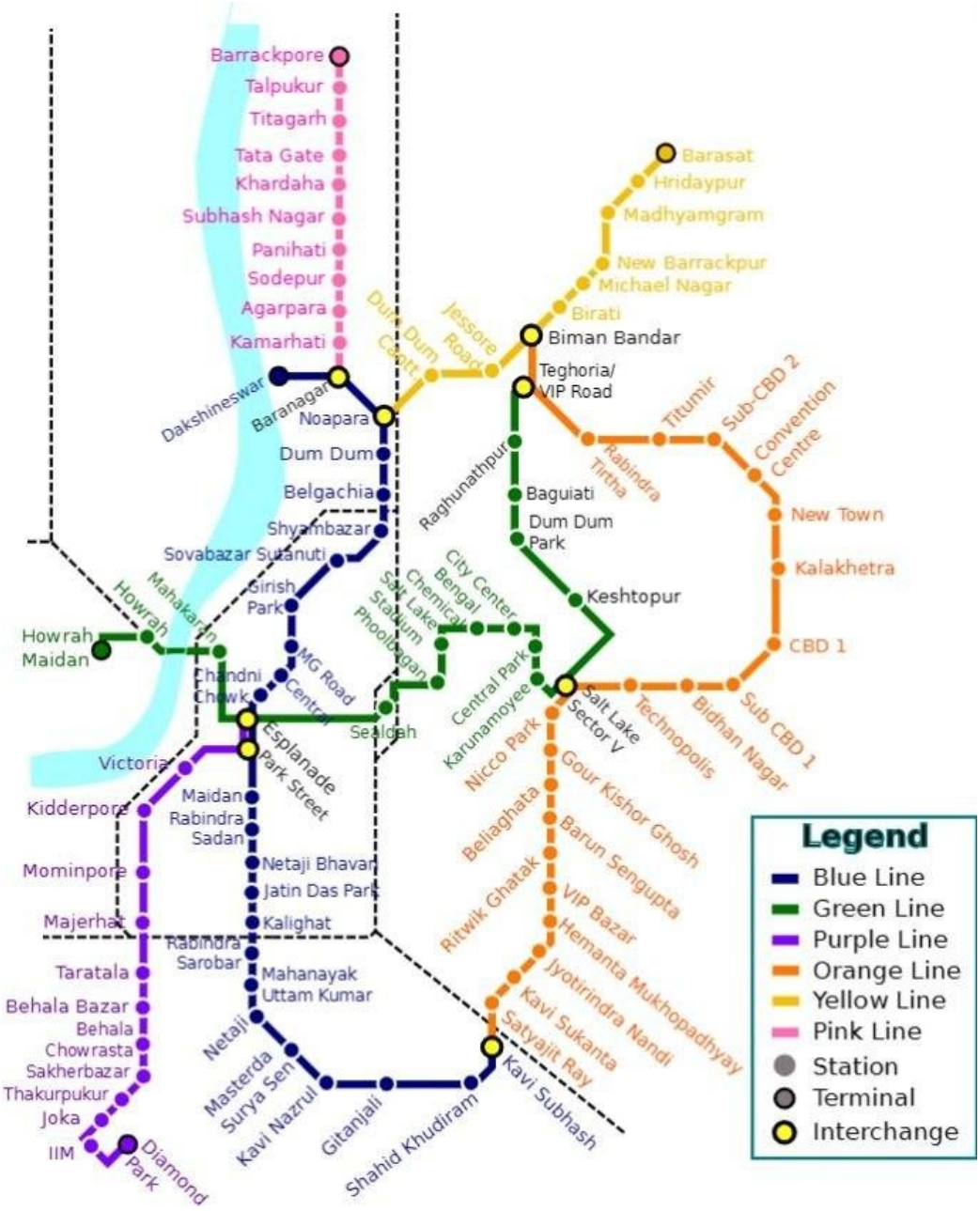


Figure 3.1: Kolkata Metro network map supplied with the project material.

3.5 Design Selection

The project adopts a combined approach: C++ is used to demonstrate graph structures and route algorithms, while HTML, CSS and JavaScript provide the browser-based journey planner. CSV files act as a lightweight data source shared conceptually across the implementation. This design was selected because it balances algorithmic clarity, a usable interface and simple dataset maintenance.

3.6 Implementation Plan / Methodology

1. Read the station, line, connection and interchange CSV files.
2. Create a graph node for each unique station-line pair.
3. Add bidirectional edges for normal metro connections with distance and travel-time weights.

4. Add directed interchange edges with zero distance, transfer time and an interchange marker.
5. Run Dijkstra with travel time as the cost for the fastest route.
6. Run lexicographic Dijkstra with (number of interchanges, travel time) as the cost for the fewest-interchange route.
7. Reconstruct the selected path using parent pointers.
8. Calculate journey totals and display the station sequence and interchange count.

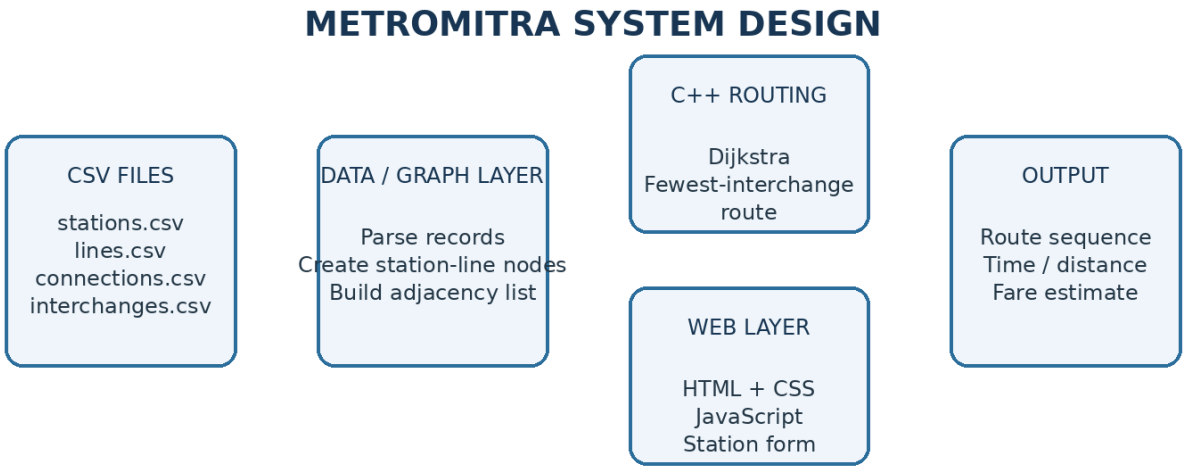


Figure 3.2: High-level architecture of the MetroMitra prototype.

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The C++ program defines structures for Station, Line, Connection, Interchange, Edge and Node. The Graph class stores station-line nodes and an adjacency list. The program reads the four CSV files, creates unique station-line nodes from the connection records, adds normal connection edges in both directions and adds directed interchange edges.

The original fastest-route method uses a priority queue and Dijkstra's algorithm. The contribution version adds an isInterchange flag to each Edge. Normal travel edges are marked false and transfer edges are marked true. The new fewestTransfersPath method stores a pair of costs for each node: (interchange count, travel time). Standard pair comparison gives lexicographic ordering, so the number of transfers is minimized first and travel time second.

The JavaScript website independently loads the CSV files, builds a graph and runs a fastest-route search. It provides station suggestions, swap controls, quick-pick journeys, line chips, route stops, distance, time and estimated fare. The supplied web source does not yet include the fewest-interchange selection; that algorithm is present in the C++ contribution version.

Dataset Summary

Dataset	Records	Fields / purpose
stations.csv	55	station_id, station_name
lines.csv	5	line_id, line_name, line_color
connections.csv	52	from/to station IDs, line ID, distance and travel time
interchanges.csv	4	station ID, from/to line IDs and transfer time

	station_id	station_name
2	S001	Dakshineswar
3	S002	Baranagar
4	S003	Noapara
5	S004	Dum Dum
6	S005	Belgachia
7	S006	Shyambazar
8	S007	Shobhabazar Sutanuti
9	S008	Girish Park
10	S009	Mahatma Gandhi Road
11	S010	Central
12	S011	Chandni Chowk
13	S012	Esplanade
14	S013	Park Street
15	S014	Maidan
16	S015	Rabindra Sadan
17	S016	Netaji Bhawan

Figure 4.1: Station master records (sample rows).

	line_id	line_name	line_color
1	L1	Blue Line	Blue
2	L2	Green Line	Green
3	L3	Purple Line	Purple
4	L4	Yellow Line	Yellow
5	L5	Orange Line	Orange

Figure 4.2: Metro line records.

	from_station_id	to_station_id	line_id	distance_km	travel_time_min
1	S001	S002	L1	1.4	3
2	S002	S003	L1	1.1	3
3	S003	S004	L1	1.8	4
4	S004	S005	L1	1.3	3
5	S005	S006	L1	1.5	3
6	S006	S007	L1	0.9	2
7	S007	S008	L1	1.2	3
8	S008	S009	L1	1.4	3
9	S009	S010	L1	1.0	2
10	S010	S011	L1	1.6	4
11	S011	S012	L1	1.3	3
12	S012	S013	L1	1.1	3

Figure 4.3: Station-to-station connection records (sample rows).

	station_id	from_line_id	to_line_id	transfer_time_min
2	S003	L1	L4	5
3	S003	L4	L1	5
4	S012	L1	L2	5
5	S012	L2	L1	5

Figure 4.4: Directional interchange records.

The four interchange rows represent two physical interchange locations with a direction-specific record in each direction. The static network map includes a Pink Line in its legend, whereas the supplied project dataset and application summary contain five lines (Blue, Green, Purple, Yellow and Orange). The dataset, rather than the map legend, is the source used by the implementation.

4.1 Implementation of Solution (Continue)

Console Execution Results

The supplied console screenshots show two example journeys using the original fastest-route functionality. A separate execution of the contribution source was also performed using the supplied CSV files. The C++17 compilation completed without compiler errors.

```
===== PLAN YOUR JOURNEY =====
Enter Source Station: Mahatma Gandhi Road
Enter Destination Station: Bengal Chemical

Source ID: S009
Destination ID: S032

===== FASTEST ROUTE =====
Mahatma Gandhi Road -> Central -> Chandni Chowk -> Esplanade -> Esplanade -> Sealdah -> Phoolbagan -> Salt Lake Stadium -
> Bengal Chemical

===== JOURNEY DETAILS =====
Total Distance: 11.326 km
Total Travel Time: 31 minutes
Estimated Ticket Fare: Rs. 30

===== LINE & INTERCHANGE DETAILS =====
Start Line: Blue Line
Interchange at: Esplanade
Continue on: Green Line
```

Figure 4.5: Console output – Mahatma Gandhi Road to Bengal Chemical.

```
===== PLAN YOUR JOURNEY =====
Enter Source Station: Salt Lake Stadium
Enter Destination Station: Dum Dum

Source ID: S031
Destination ID: S004

===== FASTEST ROUTE =====
Salt Lake Stadium -> Phoolbagan -> Sealdah -> Esplanade -> Esplanade -> Chandni Chowk -> Central -> Mahatma Gandhi Road ->
Girish Park -> Shobhabazar Sutanuti -> Shyambazar -> Belgachia -> Dum Dum

===== JOURNEY DETAILS =====
Total Distance: 16.836 km
Total Travel Time: 43 minutes
Estimated Ticket Fare: Rs. 45

===== LINE & INTERCHANGE DETAILS =====
Start Line: Green Line
Interchange at: Esplanade
Continue on: Blue Line
```

Figure 4.6: Console output – Salt Lake Stadium to Dum Dum.

```
===== PLAN YOUR JOURNEY =====
Enter Source Station: Enter Destination Station:
Source ID: S001
Destination ID: S026

===== FASTEST ROUTE =====
Dakshineswar -> Baranagar -> Noapara -> Dum Dum -> Belgachia -> Shyambazar -> Shobhabazar Sutanuti
-> Girish Park -> Mahatma Gandhi Road -> Central -> Chandni Chowk -> Esplanade -> Esplanade ->
Mahakaran -> Howrah -> Howrah Maidan

===== JOURNEY DETAILS =====
Total Distance: 17.357 km
Total Travel Time: 45 minutes
Estimated Ticket Fare: Rs. 45

===== LINE & INTERCHANGE DETAILS =====
Start Line: Blue Line
Interchange at: Esplanade
Continue on: Green Line


===== FEWEST-INTERCHANGE ROUTE =====
Dakshineswar -> Baranagar -> Noapara -> Dum Dum -> Belgachia -> Shyambazar -> Shobhabazar Sutanuti
-> Girish Park -> Mahatma Gandhi Road -> Central -> Chandni Chowk -> Esplanade -> Mahakaran ->
Howrah -> Howrah Maidan
Interchanges: 1
Travel Time: 45 minutes
Distance: 17.357 km
```

Figure 4.7: Verified C++17 execution – fastest and fewest-interchange route for Dakshineswar to Howrah Maidan.

Website Results

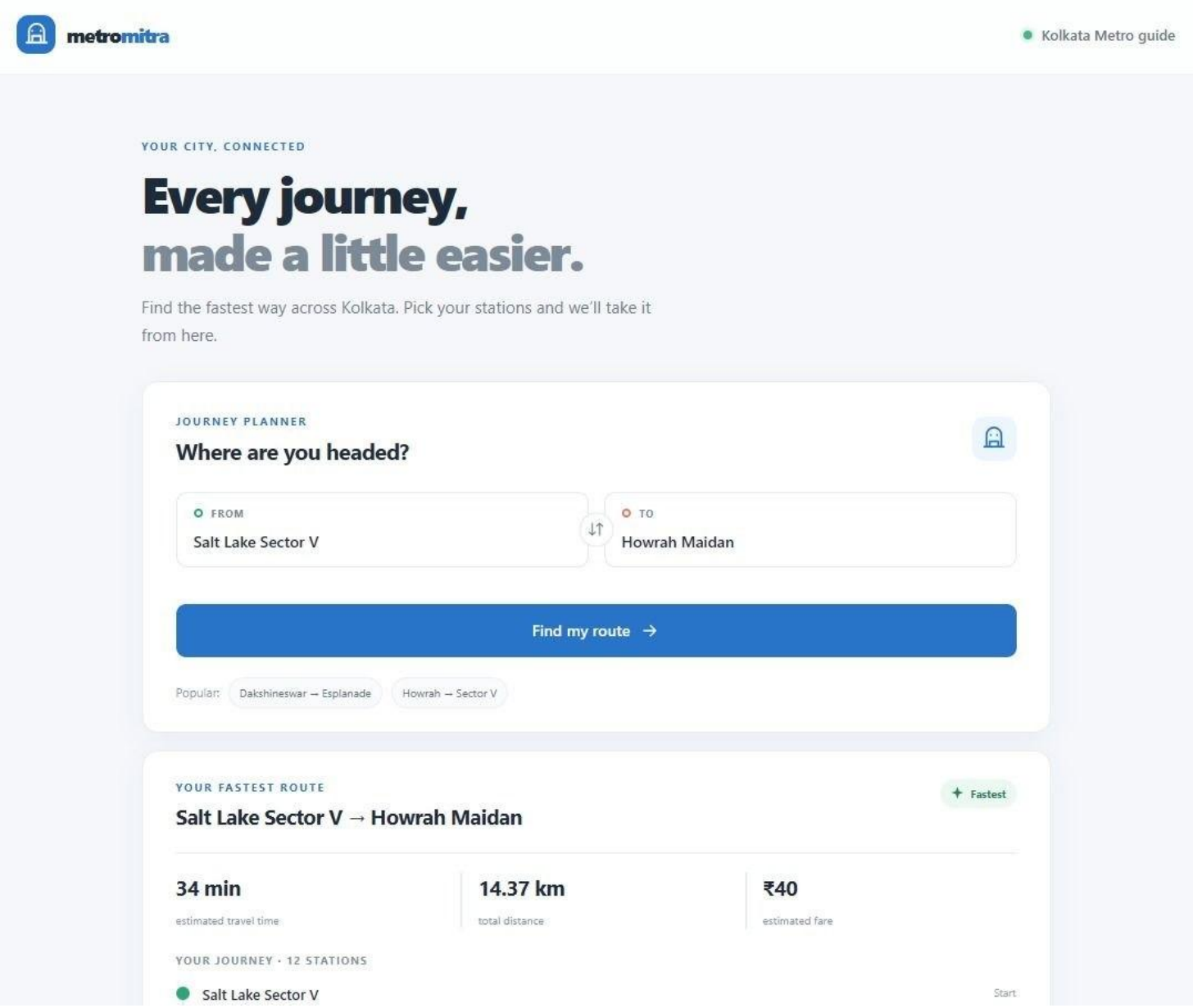


Figure 4.8: MetroMitra journey planner with a sample route result.

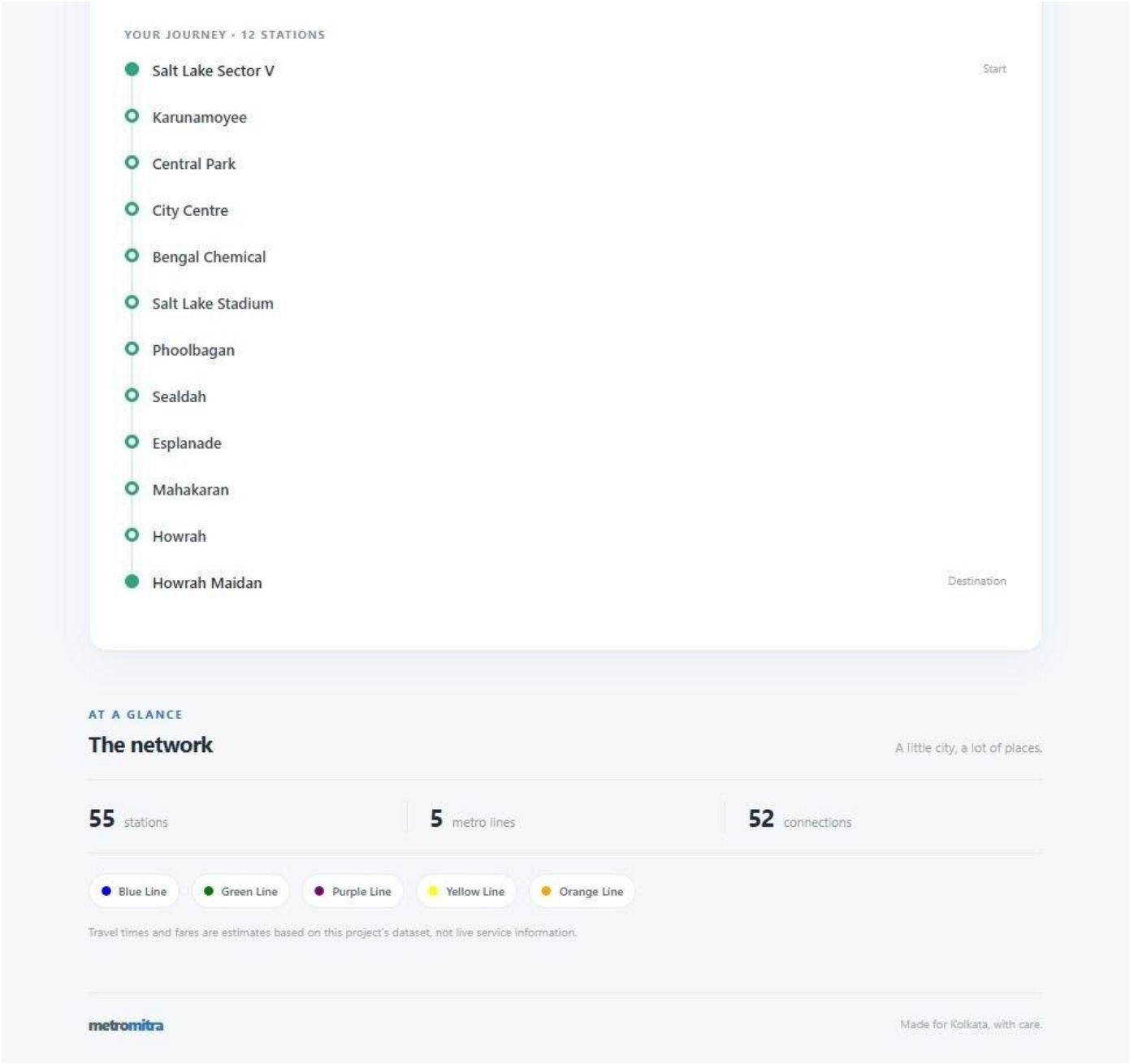


Figure 4.9: Website route-stop presentation and network summary.

4.2 Testing and Validation

Test ID	Input / condition	Expected behaviour	Observed result
TC-01	Dakshineswar → Howrah Maidan	Return a connected fastest route and journey totals	Observed: 17.357 km, 45 minutes, estimated fare ₹45; one interchange.
TC-02	Mahatma Gandhi Road → Bengal Chemical	Return a connected route and journey details	Provided terminal screenshot shows 11.326 km, 31 minutes, estimated fare ₹30.
TC-03	Salt Lake Stadium → Dum Dum	Return a connected route and journey details	Provided terminal screenshot shows 16.836 km, 43 minutes, estimated fare ₹45.
TC-04	Same source and destination	Display already-at-destination message	Validation branch exists in source; test manually before final submission.
TC-05	Unknown station name	Display invalid station message	Validation branch exists in source; test manually before final submission.
TC-06	Fewest-interchange route: Dakshineswar → Howrah Maidan	Minimize transfers, then travel time	Verified output: one interchange, 45 minutes, 17.357 km.

The contribution source was compiled with: g++ -std=c++17 main_contribution_test.cpp -o MetroMitra. It executed successfully against the supplied datasets for the sample route in TC-06. The terminal screenshots for TC-02 and TC-03 were supplied as project evidence; they are not claimed as fresh runs of the modified source.

Complexity Analysis

Operation	Time complexity	Explanation
Graph construction	O(V + E) expected with indexed lookup; current C++ node lookup may add linear scans	Builds station-line nodes and adjacency lists.
Original Dijkstra	O((V + E) log V) with a binary heap	Priority queue selects the smallest tentative travel time.
Fewest-interchange search	O((V + E) log V) with a binary heap	Each node is ordered by transfer count, then travel time.
Path reconstruction	O(V)	Follows parent pointers from destination to source.

Here V is the number of station-line graph nodes and E is the number of graph edges. The actual implementation uses vectors and linear getNodeIndex scans while building the graph; therefore, the construction phase can be slower than the ideal indexed bound. The dataset is small, so this is acceptable for the academic prototype.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

MetroMitra demonstrates how a metro network can be modelled as a weighted graph and queried using shortest-path algorithms. The project combines CSV-based data management, C++ structures and classes, adjacency lists, priority queues, Dijkstra's algorithm and a browser-based interface. It produces station sequences and estimated journey time, distance and fare.

The contribution version extends the routing objective by minimizing interchange count first and travel time second. The modified source compiled successfully under C++17 and returned a valid sample route using the supplied dataset. The output for Dakshineswar to Howrah Maidan reported one interchange, 45 minutes and 17.357 km. The result demonstrates the additional algorithmic capability, although broader route-pair testing is still recommended.

The expected outcome was a useful academic route-planning prototype. The main deviation from a production journey planner is the use of static, limited data and simulated fare rules. The system does not provide real-time service information, and the web interface has not yet been extended to expose the new fewest-interchange option.

5.2 Future Work

- Integrate the fewest-interchange algorithm into the JavaScript web interface and allow users to choose a routing preference.
- Expand and verify the station, connection and interchange dataset against reliable current sources.
- Replace the simulated fare formula with an official fare table, if an authoritative source and permitted data are available.
- Add alternative-route comparison showing time, distance and number of transfers side by side.
- Add timetable, waiting-time and service-disruption information only when reliable data feeds are available.
- Improve CSV parsing to handle quoted fields, malformed rows and missing values robustly.
- Use a hash map from station-line keys to node indices to reduce graph-construction lookup time.
- Add automated unit tests and test many connected, disconnected, same-station and invalid-input cases.
- Improve accessibility, mobile responsiveness and user guidance in the web interface.

REFERENCES

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.

[3] ISO/IEC 14882:2017, Programming Languages – C++.

[4] T. Berners-Lee, R. Fielding and L. Masinter, RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files, IETF, 2005.

[5] WHATWG, HTML Living Standard, https://html.spec.whatwg.org/ (accessed November 2026).

[6] ECMA International, ECMAScript Language Specification, https://tc39.es/ecma262/ (accessed November 2026).

[7] MetroMitra project source files: main.cpp, app.js, index.html, styles.css and the four CSV datasets, supplied as project materials.

[8] MetroMitra – Kolkata Metro Navigation System, project README and repository description, https://github.com/anuskadatta28-tech/MetroMitra-Kolkata-Metro-Navigation-System.

APPENDIX

A. Project File Structure

```
MetroMitra-Kolkata-Metro-Navigation-System/
├── dataset/
│   ├── stations.csv
│   ├── lines.csv
│   ├── connections.csv
│   └── interchanges.csv
├── screenshots/
├── src/
│   └── main.cpp
├── web/
│   ├── README.md
│   ├── app.js
│   ├── index.html
│   └── styles.css
├── README.md
└── netlify.toml
```

B. Design Checklist

☐ Problem and intended users identified.

☐ Dataset entities and relationships documented.

☐ Graph representation and edge weights specified.

☐ Fastest-route algorithm implemented.

☐ Fewest-interchange extension added in contribution source.

☐ Sample execution compiled and checked.

☐ Limitations and simulated fare clearly disclosed.

☐ Screenshots placed with captions.

☐ User manual and references included.

C. Notes for Final Submission

The supervisor and Head of Department names/signatures, viva date, plagiarism report and any department-specific design checklist evidence must be completed by the project team or department. They have not been invented in this draft.

USER MANUAL

This manual describes how to run the C++ console program and view the browser interface. Keep the dataset folder in the project root because the C++ program opens files using relative paths such as dataset/stations.csv.

A. Requirements

- Windows, Linux or macOS computer.
- GCC g++ compiler with C++17 support for the C++ contribution build.
- A modern web browser for the web interface.
- The complete MetroMitra project folder, including dataset and web subfolders.

B. Run the C++ Console Program

1. Open the project folder in Visual Studio Code or a terminal.
2. Confirm that the dataset folder contains stations.csv, lines.csv, connections.csv and interchanges.csv.
3. Open a terminal at the project root directory.
4. Compile the contribution source using: g++ -std=c++17 src/main.cpp -o MetroMitra (use the final modified main.cpp filename and path).
5. On Windows, run .\MetroMitra.exe. On Linux/macOS, run ./MetroMitra.
6. Enter the exact source station name when prompted.
7. Enter the exact destination station name when prompted.
8. Read the fastest route, journey details, line/interchange details and fewest-interchange route output.

C. Run the Web Interface

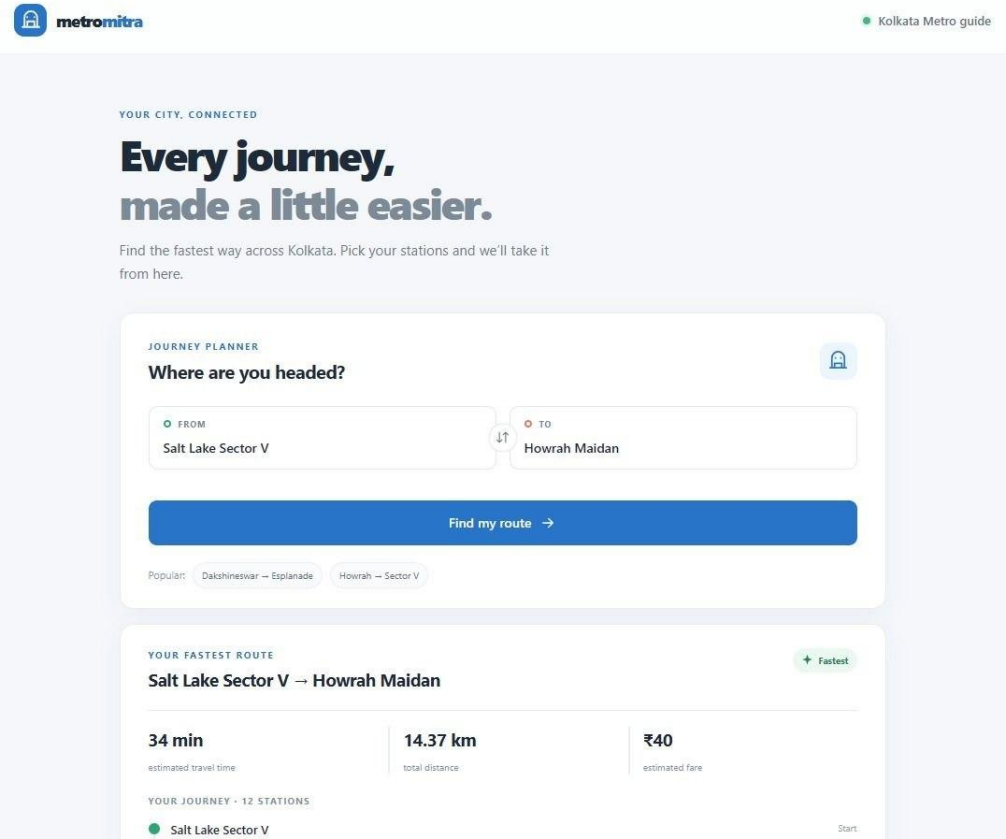
9. Open a terminal at the project root.
10. Start a local static web server; for example, with Python installed, run: python -m http.server 8000.
11. Open http://localhost:8000/web/ in a browser.
12. Enter or select a source station and destination station from the suggestions.
13. Select Find my route to display the fastest route.
14. Use the swap control to reverse the source and destination if needed.
15. Review the route sequence, estimated time, distance and fare.

The JavaScript loads CSV files using fetch, so opening the HTML file directly with a file:// URL may prevent dataset loading in some browsers. A local HTTP server is recommended.

D. Input Guidance and Limitations

Use station names exactly as they appear in stations.csv. The current web interface provides station suggestions. The fare is an academic estimate, not an official fare. Routes are limited to the supplied dataset and do not include live service updates. The fewest-interchange feature is verified in the C++ contribution source; the current supplied web JavaScript presents the fastest route only.

E. Evidence Screenshots



User manual Figure U.1: Browser journey planner and sample route result.

```
===== PLAN YOUR JOURNEY =====
Enter Source Station: Enter Destination Station:
Source ID: S001
Destination ID: S026

===== FASTEST ROUTE =====
Dakshineswar -> Baranagar -> Noapara -> Dum Dum -> Belgachia -> Shyambazar -> Shobhabazar Sutanuti
-> Girish Park -> Mahatma Gandhi Road -> Central -> Chandni Chowk -> Esplanade -> Esplanade ->
Mahakaran -> Howrah -> Howrah Maidan

===== JOURNEY DETAILS =====
Total Distance: 17.357 km
Total Travel Time: 45 minutes
Estimated Ticket Fare: Rs. 45

===== LINE & INTERCHANGE DETAILS =====
Start Line: Blue Line
Interchange at: Esplanade
Continue on: Green Line


===== FEWEST-INTERCHANGE ROUTE =====
Dakshineswar -> Baranagar -> Noapara -> Dum Dum -> Belgachia -> Shyambazar -> Shobhabazar Sutanuti
-> Girish Park -> Mahatma Gandhi Road -> Central -> Chandni Chowk -> Esplanade -> Mahakaran ->
Howrah -> Howrah Maidan
Interchanges: 1
Travel Time: 45 minutes
Distance: 17.357 km
```

User manual Figure U.2: Console output showing fastest and fewest-interchange route summaries.